Smart Finance: Design and Preliminary Evaluation of an AI-Assisted Personal Finance Application

Transaction Planning, Expense Analysis and Receipt Extraction

Kaber Daryn, Nuriman Aisultan, Akhmetov Dauren

School of Software Engineering, Astana IT University, Astana, Kazakhstan
Software Engineering programme, group SE-2430
Supervisor: Bakayeva Bakyt, Senior Lecturer

Abstract

Smart Finance is a prototype for recording personal transactions, reviewing expenses and setting budgets and savings goals. It combines a Flutter client, a Python API and relational storage with optional category suggestions, a simple expense forecast and externally hosted receipt extraction. We describe the implementation and evaluate receipt processing on a convenience sample of 16 unique images supplied by a team member and friends. Using gemini-3.8-flash through the Gemini provider, the recorded run matched 10/15 merchant references, 9/13 purchase-date references and 14/16 receipt totals. Two requests failed because the provider quota was exhausted; these failures remain in the primary denominators. Mean processing time was 29.49 seconds. The results motivate editable extraction and a manual-entry fallback in a personal finance workflow.

Keywords: personal finance; expense analysis; receipt extraction; Flutter; human-in-the-loop systems.

1. Introduction

A personal finance diary is useful only when its records are sufficiently complete and understandable. Manual entry gives users control, but repeating amounts, dates and categories creates opportunities for omissions and typing errors. Financial-literacy research motivates tools that support informed financial decisions [1].

We investigate the engineering question: how can transaction capture, spending summaries and planning tools be combined with assistance that remains editable? The contribution is an integrated prototype and a transparent preliminary evaluation of its receipt-processing pipeline, with editable analytical assistance. Here, AI-assisted refers to external multimodal receipt extraction and baseline category suggestions, not to a new learning algorithm or an autonomous financial adviser.

2. Related Work and Design Rationale

Human–AI interaction guidance recommends making incorrect suggestions easy to correct or dismiss [2]. We apply this principle by requiring users to review extracted receipt fields before saving an expense. The extraction service therefore assists data entry rather than replacing user control.

We use multinomial Naive Bayes as an inspectable baseline for short expense descriptions [3], and a three-month moving average as a transparent initial forecast. Simple forecasting methods provide useful benchmarks for later comparisons [4]. Neither component represents a novel algorithm.

SROIE distinguishes receipt text localisation, OCR and key-information extraction [5]. We instead delegate image interpretation to a multimodal Gemini model with documented image-input support [6]. Our local receipt pilot evaluates application-level field extraction, not performance on the SROIE benchmark.

Classification proposes categories, forecasting estimates monthly spending, and receipt extraction proposes transaction fields. We evaluate these tasks separately: an extraction match is not evidence of forecasting accuracy. Our contribution is their integration with editable suggestions and explicit confirmation.

3. System Architecture

We separate clients, API, storage and analytical services (Figure 1). The Python API uses ThreadingHTTPServer and authenticated JSON endpoints for transactions, summaries, budgets and savings goals. SQLite supports local development; the source also supplies a PostgreSQL adapter and Docker Compose configuration [7].

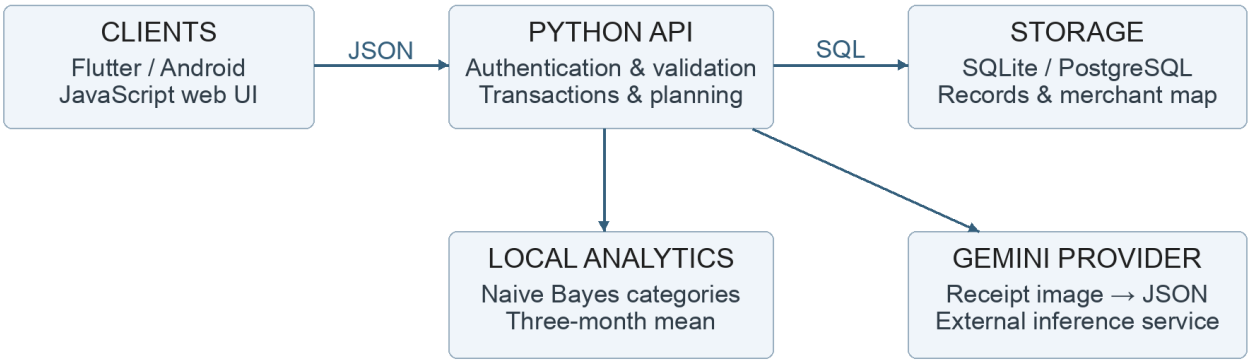


Figure 1. System architecture. Receipt extraction uses an external provider; local analytics and financial storage remain separate.

The Flutter client targets Android. For development, we also use a separate JavaScript browser client when emulator resource use is high. These clients share the API but do not expose identical features. In the receipt flow, recognition returns editable fields; an expense is stored only after explicit confirmation.

4. Transaction Handling and Intelligent Components

4.1 Financial records and planning

Amounts are stored as integer minor units of KZT, avoiding binary floating-point arithmetic in the ledger. Authenticated operations are scoped to the current user. Passwords are salted and hashed, and stored session tokens are hashed. We use this configuration for local development.

Monthly budgets compare recorded expenses with an overall or category-specific limit. Savings goals track a target and a manually entered saved amount; updating a goal does not transfer money or alter the ledger balance. Spending observations are rules about exceeded budgets and category shares, not investment advice. CSV import validates batches and detects identical content, not all semantically duplicate transactions.

4.2 Description-based category suggestions

The analytical module implements multinomial Naive Bayes with add-one smoothing [3]. Its bundled corpus contains 40 manually constructed English descriptions across 5 categories. For description d, the selected category maximises the class-prior term plus the word-count log-likelihood:

$$\hat{c} = arg\ max_c \{ \log P(c) + \Sigma_w n(w,d) \log P(w|c) \}.$$

The implementation lowercases text and tokenises Latin letters and digits. Its vocabulary is currently limited to the English training corpus. When all tokens are unfamiliar, it returns no category. Otherwise, the returned confidence is an uncalibrated model score. The user can change the suggestion before saving. We use synthetic examples for integration checks; classifier evaluation requires a separate labelled expense dataset.

4.3 Expense estimation

The forecast is the arithmetic mean of recorded expenses in the previous three completed calendar months. With monthly totals y, the estimate is:

$$\hat{y}_t = (y_{t-1} + y_{t-2} + y_{t-3}) / 3.$$

The current incomplete month is excluded. Missing months after the first expense are treated as zero, so an incomplete diary can reduce the estimate. We use this transparent baseline pending evaluation on chronological expense records [4].

4.4 Receipt extraction and confirmed merchant categories

The server sends an image to the configured Gemini provider and requests merchant, date, total and currency as JSON. It validates the response and returns uncertain or invalid fields as null. Category reuse follows a different path: a normalised merchant key retrieves that user's previously confirmed category. The mapping is updated atomically when the expense is saved. It is not online training of the Naive Bayes model. We do not implement a separate QR-code decoder.

5. Preliminary Evaluation Protocol

5.1 Materials and reference preparation

We collected 17 receipt images from a team member and friends. A byte-level duplicate check left 16 unique images. The convenience sample varies in layout and quality. We checked spreadsheet labels against the images and left unreadable fields unknown; one blurred date remains provisional. Receipt owners consented to research use and processing through the external Gemini service. Personal identifiers and payment-card details are not published.

Reference coverage comprises 15 merchants, 13 purchase dates and 16 totals. Unknown references are excluded only from that field's denominator. An old receipt outside the accepted date-age window is retained as a validation stress case. Figure 2 illustrates the review interface; it is not an additional scored receipt.

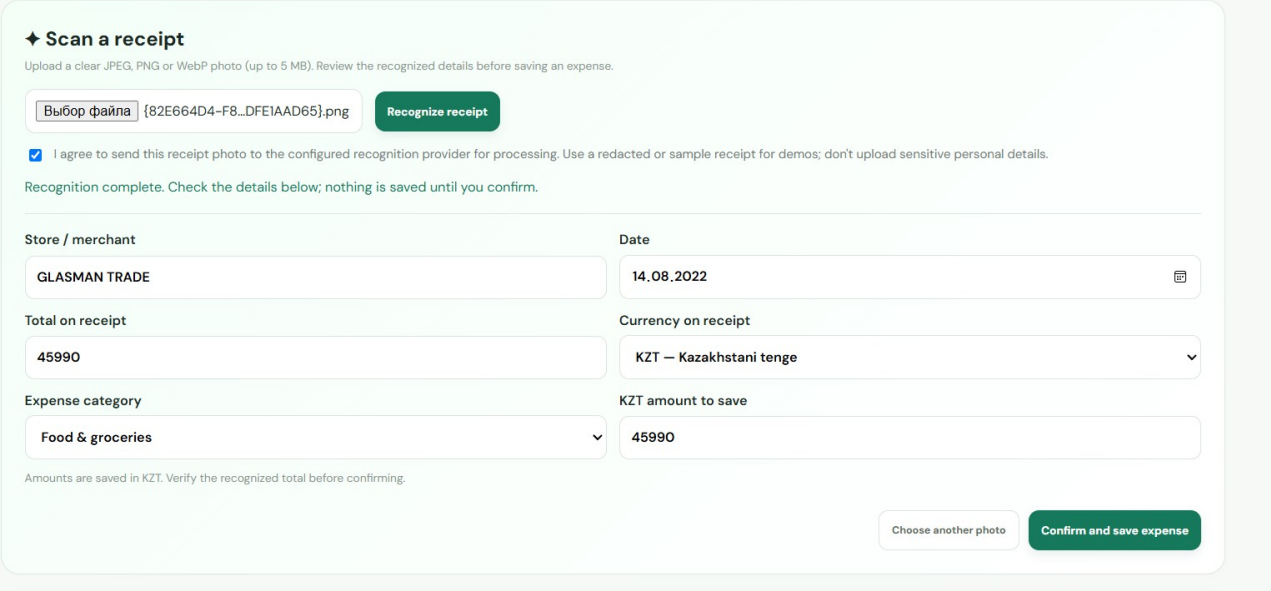


Figure 2. Browser receipt-review interface supplied by the authors. Extracted fields remain editable and are not saved until confirmation.

5.2 Procedure and matching rules

We ran backend.evaluate_receipts with provider gemini and model gemini-3.8-flash. The JSON report records one sequential attempt per image using the receipt endpoint's extraction and validation functions. We checked aggregate results against per-image flags, without selective retries. This procedure evaluates the backend pipeline rather than end-to-end Android interaction.

Merchant matching normalises case and legal-form text; it is not fuzzy semantic matching. Dates require exact ISO-date equality and totals exact decimal equality. For field f, we calculate:

$$A_f = C_f / N_f,$$

where C_f counts matched predictions and N_f counts images with a known reference. Provider failures remain in N_f as misses. Currency is not scored. Timing includes file reading, the external request and response validation; the 95th percentile uses the nearest-rank rule. The report does not record hardware, network conditions or an exact model revision.

6. Results and Interpretation

Our run returned 14 non-error pipeline responses from 16 attempts (87.5%). Both remaining attempts explicitly reported that the receipt-scanning quota was exhausted. They are service-availability failures, not evidence that the model read the images incorrectly. Nevertheless, they prevented usable extraction and remain in the primary field-match rates.

Table 1. Primary field-match results, including provider failures.

Field	Matched / labelled	Match rate
Merchant	10 / 15	66.67%
Purchase date	9 / 13	69.23%
Total	14 / 16	87.50%

The denominators differ because reference coverage differs. Among the 14 non-error responses, 3 labelled merchant fields and 2 labelled dates did not match. All 14 returned totals matched their references. That conditional observation does not replace the primary total match rate of 87.50%, which includes the quota failures.

The report retains booleans and sanitised errors, not extracted field values. It therefore cannot show whether a particular merchant mismatch arose from spelling, a brand/legal-name alternative or another response. Likewise, the old receipt's date mismatch is consistent with the application's age validation, but the raw model response is unavailable to establish the exact cause. The uncertain blurred-date reference further prevents treating every date mismatch as a confirmed recognition error.

Table 2. Processing time for all attempts, including quota failures.

Statistic	Seconds
Mean	29.49
Median	29.56
95th percentile	66.02

The two quota failures returned quickly, reducing the all-attempt mean. Mean time among non-error responses was 33.61 seconds. With only 16 observations, the nearest-rank 95th percentile is the maximum observed value, not a stable tail estimate. We therefore retain visible progress, recoverable failure messages and manual entry.

We also ran the source snapshot's Python test suite locally: it passed 50 tests against temporary SQLite databases and local HTTP servers. Those tests cover API and data-handling behaviour, including mocked receipt responses. They do not measure real-provider accuracy, production load, PostgreSQL deployment reliability or native Android usability.

7. Discussion and Conclusion

Our contribution is an integrated personal finance prototype that combines transaction recording, budgets, savings goals and editable AI-assisted capture. Receipt extraction may reduce repetitive typing, but whether it reduces overall data-entry effort compared with manual entry requires a separate comparative study. Merchant and date mismatches justify keeping user confirmation. Quota failures also show why external-provider availability must be handled independently of recognition quality.

The study is limited by a small convenience sample, one recorded run, strict name matching and one uncertain date reference. We did not measure usability, financial outcomes or native Android performance. The browser client remains a development interface: static dashboard indicators and receipt-prefill rounding require correction before a deployment evaluation. These interface issues do not alter the direct backend evaluation reported here.

The present empirical results concern receipt extraction only. Planned evaluations include category accuracy, macro-F1 and forecasting MAE/MAPE; these have not been measured in the present receipt study. That phase requires independently labelled expense descriptions and chronological personal-expense records, with held-out testing and a simple forecasting benchmark. MAPE additionally requires an explicit rule for zero-expense months.

We conclude that a human-in-the-loop workflow is a practical design for this prototype: validate extracted fields, allow correction, confirm before saving and preserve manual entry. Our next steps are to correct client inconsistencies, resolve uncertain labels and evaluate classification, forecasting and usability on suitable data.

Author Contributions

Kaber Daryn: mobile client, backend API, system architecture and integration.

Nuriman Aisultan: UX/UI design, test planning and system testing.

Akhmetov Dauren: analytical module, expense categorisation, forecasting and recommendation logic.

References

[1] Lusardi, A., and Mitchell, O. S. (2014). The Economic Importance of Financial Literacy: Theory and Evidence. Journal of Economic Literature, 52(1), 5–44. https://doi.org/10.1257/jel.52.1.5

[2] Amershi, S., et al. (2019). Guidelines for Human-AI Interaction. Proceedings of the CHI Conference on Human Factors in Computing Systems, 13 pp. https://doi.org/10.1145/3290605.3300233

[3] McCallum, A., and Nigam, K. (1998). A Comparison of Event Models for Naive Bayes Text Classification. AAAI-98 Workshop on Learning for Text Categorization. https://aaai.org/papers/041-ws98-05-007

[4] Hyndman, R. J., and Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd ed. OTexts, Section 5.2. https://otexts.com/fpp3/simple-methods.html

[5] Huang, Z., Chen, K., He, J., Bai, X., Karatzas, D., Lu, S., and Jawahar, C. V. (2019). ICDAR2019 Competition on Scanned Receipt OCR and Information Extraction. Proceedings of ICDAR, 1516–1520. https://doi.org/10.1109/ICDAR.2019.00244

[6] Google. (2026). Gemini 3.8 Flash. Gemini API developer documentation (accessed 24 October 2026). https://ai.google.dev/gemini-api/docs/models/gemini-3.8-flash

[7] Smart Finance project team. (2026). Smart Finance source archive, snapshot 8f4a9c227a67b6bbfe9a1d33df3e2c18261097d0. Software supplied for this study. https://github.com/KaberDaryn/smart-finance-project